

CSA05 – Database Management Systems

CO4: Industry Problem Solving Task – Solutions

1. E-Commerce Company – File Organization for 10 Million Product Records

Recommended Strategy: Clustered B+ Tree Indexed File Organization

For an e-commerce company storing **10 million product records**, a **B+ Tree-based indexed file organization** is recommended.

Justification

Products are frequently searched using Product ID, category, price, and other attributes. A B+ Tree provides efficient searching while also supporting insertions, deletions, and range queries.

Assume:

- Number of records = 10,000,000
- Average record size = 200 bytes
- Block size = 4 KB = 4096 bytes

Records per block:

A sequential scan may require up to **500,000 block accesses**.

With a B+ Tree, the tree height is usually around 3–4 levels because each index node can contain many pointers. Therefore:

- Search cost $\approx$ **3–4 I/O operations**
- Insert cost $\approx$ **3–4 I/O operations**
- Delete cost $\approx$ **3–4 I/O operations**

Conclusion

A **clustered B+ Tree index on ProductID** is the best primary organization because it provides low search cost and supports dynamic updates. Secondary B+ Tree indexes can be created for attributes such as Category and Price.

2. Banking System – Indexing by Account Number and Date Range

Recommended Strategy: Composite B+ Tree Index

The banking system needs:

1. Fast retrieval using Account Number.
2. Range retrieval using transaction dates.

A composite index can be created as:

(AccountNumber, TransactionDate)

Why B+ Tree?

For a query such as:

```
SELECT *  
FROM Transactions  
WHERE AccountNumber = 1001  
AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31';
```

The B+ Tree first locates AccountNumber 1001 and then efficiently scans the linked leaf nodes for the required date range.

Recommended Indexes

- **Primary/Clustered B+ Tree:** (AccountNumber, TransactionDate)
- **Optional Secondary Index:** TransactionID

Conclusion

A composite **B+ Tree index on (AccountNumber, TransactionDate)** provides efficient point lookup and range queries, with approximately **$O(\log N + K)$** complexity, where K is the number of matching transactions.

3. Healthcare System – Multi-Level Indexing

The healthcare system accesses records using:

- PatientID
- Doctor Name

Recommended Design

Use a **primary index on PatientID** and a **secondary multi-level B+ Tree index on DoctorName**.

Primary Index

PatientID → Block Address

Example:

PatientID	Block Address
P1001	Block 10
P1002	Block 10
P1003	Block 11

Secondary Index

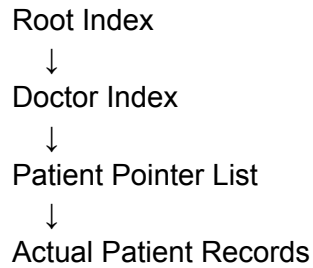
DoctorName → List of Patient Record Pointers

Example:

Doctor Name	Patient Records
Dr. Kumar	P1001, P1020, P1055
Dr. Ravi	P1002, P1010
Dr. Priya	P1003, P1008

Multi-Level Structure

If the secondary index becomes large, another index level is created:



Conclusion

A **multi-level B+ Tree indexing system** provides scalable access for both PatientID and DoctorName searches.

4. Logistics Company – B-Tree vs B+ Tree

Requirement

The company performs frequent:

- Shipment insertions
- Shipment deletions
- Searches
- Tracking queries

Comparison

Feature	B-Tree	B+ Tree
Data stored	Internal and leaf nodes	Leaf nodes only
Search	Fast	Fast
Range queries	Less efficient	Highly efficient
Sequential traversal	Moderate	Excellent
Leaf linking	No	Yes
Fan-out	Lower	Higher

Tree height	Higher	Lower
-------------	--------	-------

Recommendation: B+ Tree

B+ Tree is more suitable because:

1. All actual records are stored in leaf nodes.
2. Leaf nodes are linked for efficient sequential access.
3. Higher fan-out reduces tree height.
4. Dynamic insertions and deletions are efficiently managed through splitting and merging.
5. Shipment records can be searched by tracking number and scanned sequentially.

Conclusion

For a logistics system with frequent dynamic updates, **B+ Tree is preferred over B-Tree** because it provides better storage utilization and efficient range/sequential operations.

5. HR System – Hashing Scheme for 5000 Employees

Recommended Hash Key

Use:

EmployeeID

Example hash function:

```
[
h(EmployeeID) = EmployeeID \bmod 1000
]
```

Static Hashing

Assume 1000 buckets:

Bucket = EmployeeID % 1000

Advantages:

- Simple implementation.
- Fast direct access.
- Low overhead.

Disadvantages:

- Fixed number of buckets.
- Overflow chains may increase as employees grow.
- Reorganization is difficult.

Extendible Hashing

Extendible hashing uses:

- Global depth.
- Local depth.
- Directory.
- Bucket splitting.
- Directory doubling.

Advantages:

- Dynamically grows with employee records.
- Reduces overflow chains.
- Maintains near-constant search performance.

Comparison

Feature	Static Hashing	Extendible Hashing
Number of buckets	Fixed	Dynamic
Growth support	Poor	Excellent
Overflow handling	Chaining	Bucket splitting
Search performance	$O(1)$ average	$O(1)$ average
Storage overhead	Low	Higher
Scalability	Limited	High

Conclusion

For exactly 5000 relatively stable employee records, **static hashing is sufficient**. However, if the organization expects significant employee growth and frequent updates, **extendible hashing is the better solution**.

6. Airline Reservation System – Combined Index Structure

Requirements

The system must support:

- Point queries by FlightNumber.
- Range queries by Date.

Recommended Structure

Use two indexes:

Primary Hash Index on FlightNumber

Secondary B+ Tree Index on FlightDate

Hash Index

FlightNumber → Flight Record

Example:

AI101 → Record

6E234 → Record

UK567 → Record

average lookup time.

B+ Tree Index

FlightDate



B+ Tree



Linked Leaf Nodes

Example query:

```
SELECT *  
FROM Flights  
WHERE FlightDate BETWEEN '2026-08-01' AND '2026-08-10';
```

The B+ Tree efficiently retrieves all records in the date range.

Conclusion

The combined structure is:

FlightNumber → Hash Index → Point Queries

FlightDate → B+ Tree → Range Queries

This provides efficient performance for both query types.

7. University Student Database – File Organization and Secondary Indexing

Requirements

The system needs fast queries using:

- Department
- CGPA range

Recommended File Organization

Store the main student file using a **clustered B+ Tree based on StudentID**.

Create a secondary composite index:

(Department, CGPA)

Example

Student Records:

StudentID	Department	CGPA
101	CSE	8.5
102	IT	7.8
103	CSE	9.1

104	ECE	8.2
-----	-----	-----

Secondary index:

(CSE, 8.5) → Student 101

(CSE, 9.1) → Student 103

(ECE, 8.2) → Student 104

(IT, 7.8) → Student 102

Query

```
SELECT *
FROM Students
WHERE Department = 'CSE'
AND CGPA BETWEEN 8.0 AND 9.5;
```

The B+ Tree first locates the CSE entries and then performs an efficient range scan on CGPA.

Conclusion

Use:

Primary Organization: StudentID-based clustered file

Secondary Index: B+ Tree on (Department, CGPA)

This supports efficient departmental filtering and CGPA range queries.

8. Disk Block Utilization – Heap, Sorted and Hashed Files

Given

- Number of records = 100,000
- Record size = 50 bytes
- Block size = 4 KB = 4096 bytes

Step 1: Calculate Records per Block

$$50/4096=81.92$$

Therefore, each block can store:

81 records

Space used:

$$81 \times 50 = 4050 \text{ bytes}$$

Unused space per block:

$$4096 - 4050 = 46 \text{ bytes}$$

Step 2: Number of Blocks

$$81100,000 = 1234.57$$

Therefore:

1235 blocks

Comparison

Organization	Blocks Required	Utilization
Heap File	1235	High
Sorted File	1235	High
Hashed File	Approximately 1235+	Depends on overflow

Heap File

Records are inserted wherever free space exists.

Utilization is approximately:

$$40964050 \times 100 = 98.88\%$$

Sorted File

The basic block utilization is also approximately **98.88%**. However, updates may require shifting records or maintaining free space, which can reduce effective utilization.

Hashed File

Primary buckets may have high utilization, but collisions can create overflow blocks. Therefore, actual disk utilization may be lower.

Conclusion

- **Heap:** Best for simple insertions and good block utilization.
 - **Sorted:** Good utilization but expensive updates.
 - **Hashed:** Fast equality search but overflow may increase disk usage.
-

9. Social Media Platform – Composite Indexing for 500 Million Posts

Requirements

The platform indexes:

- `user_id`
- `timestamp`
- `hashtag`

Recommended Index Strategy

Use multiple specialized indexes.

Index 1: Composite B+ Tree

(`user_id`, `timestamp`)

This supports queries such as:

```
SELECT *  
FROM Posts  
WHERE user_id = 1001  
AND timestamp BETWEEN '2026-08-01' AND '2026-08-20';
```

The B+ Tree efficiently finds a user's posts and performs a timestamp range scan.

Index 2: Hashtag Secondary Index

hashtag → List of PostIDs

Example:

```
#AI → [P101, P220, P450]  
#Python → [P102, P300, P501]
```

For large-scale implementation, the hashtag index can use:

- Hash-based lookup for exact hashtags.
- Inverted index mapping hashtags to Post IDs.

Index 3: Primary Post Index

PostID → Actual Post Record

Overall Architecture

PostID



Primary Record Storage

(user_id, timestamp)



Composite B+ Tree



User Timeline Queries

hashtag



Hash / Inverted Index



PostID List

Conclusion

For 500 million posts:

- **B+ Tree on (user_id, timestamp)** for user timelines.
- **Inverted/hash index on hashtag** for hashtag searches.
- **Primary index on PostID** for direct record access.

This combination supports scalable point, range, and text-category queries.

10. Heap File vs Sorted File vs Hashed File – Supermarket Billing System

Scenario

The supermarket processes:

1,000,000

transactions per day.

Operations include:

- Insert new billing transactions.
- Search transactions using Bill Number.
- Delete or archive old transactions.

Comparison

Operation	Heap File	Sorted File	Hashed File
Insert	$O(1)$	$O(N)$	$O(1)$ average
Search by key	$O(N)$	$O(\log N)$	$O(1)$ average
Delete	$O(N)$	$O(N)$	$O(1)$ average
Range search	$O(N)$	$O(\log N + K)$	Poor
Sequential scan	Excellent	Excellent	Moderate

Heap File

Advantages:

- Very fast insertion.
- Simple storage management.

Disadvantages:

- Searching a particular bill may require scanning many records.
- Poor performance for 1 million transactions.

Sorted File

Advantages:

- Binary search is efficient.
- Good for range-based transaction queries.

Disadvantages:

- New transaction insertion requires maintaining sorted order.
- Frequent inserts can become expensive.

Hashed File

Use:

$\text{BillNumber} \% \text{NumberOfBuckets}$

Advantages:

- Very fast billing transaction lookup.
- Fast insertions.
- Fast deletions.

Disadvantages:

- Not suitable for range queries.
- Collision and overflow management is required.

Performance Summary

Heap File:

Insert $\rightarrow O(1)$

Search $\rightarrow O(N)$

Delete $\rightarrow O(N)$

Sorted File:

Insert $\rightarrow O(N)$

Search $\rightarrow O(\log N)$

Delete $\rightarrow O(N)$

Hashed File:

Insert $\rightarrow O(1)$ average

Search $\rightarrow O(1)$ average

Delete $\rightarrow O(1)$ average

Final Recommendation

For a supermarket billing system processing **1 million transactions per day**, a **Hashed File organization using BillNumber as the hash key** is the most suitable for operational transaction processing because the primary requirement is fast insert and exact bill-number retrieval.

If the system also requires frequent date-wise reports and range queries, an additional **B+ Tree index on TransactionDate** should be maintained.